

Action-Prefix System — Implementation Report (§11.4.140)

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Scope	the decoupled, reusable machinery implementing the §11.4.140 design
Inputs	DESIGN.md, RULE_DRAFT.md, TEST_PLAN.md, RESEARCH.md (this dir)

Anti-bluff (§11.4.6): every claim below was verified by running the code in this session, not asserted. Every script is `sh -n + bash -n clean` (§11.4.67); the expander + hook are deterministic across N=3 identical runs (§11.4.50); the awk fallback is byte-identical to the python path.

Boundary: NO live governance file (Constitution.md / CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md) was edited — those are conductor-owned (§11.4.58 contention paths). The LAYER-1 mirror block lives as canonical text in actions/recognition_instruction.md for the conductor to embed via the §11.4.26 pipeline. No commit / push / git-checkout performed.

1. Files created (absolute paths)

Path	Role	Layer
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/actions/registry.yaml	single source of truth: action → expansion + grammar + rules	shared

Path	Role	Layer
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/actions/recognition_instruction.md	the verbatim LAYER-1 recognition-instruction block (marked for embedding into the 4 carriers)	1
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/scripts/action_prefix_lib.sh	shared pure expander library (apx_* API)	2 + tests
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/scripts/hooks/action_prefix_expand.sh	Claude Code UserPromptSubmit hook (LAYER 2 mechanical)	2
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/scripts/generate_agent_prefix_commands.sh	per-agent slash-command generator (Gemini/Qwen TOML, Codex md)	2
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/scripts/install_action_prefix.sh	idempotent installer — wires the hook by reference into .claude/settings.json + runs the generator	install seam
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/actions/generated/{gemini,qwen,codex}/background.{toml,md} + README.md	generated slash-command artefacts (output of the generator)	2 (generated)
/run/media/milosvasic/DATA4TB/Projects/Android_15/constitution/docs/research/action_prefix_system/IMPLEMENTATION_REPORT.md	this report	doc

All four scripts are chmod +x.

2. How each piece composes (the two-layer system over one registry)

- **actions/registry.yaml** is the SINGLE SOURCE OF TRUTH (§11.4.140 extension contract). It carries `schema_version`, the grammar block (`prefix_regex ^([A-Z][A-Z0-9_]*) ::,` `case_sensitive: true`, `multiple_prefixes: stack`, `escape: \`) and the BACKGROUND action with the operator's verbatim expansion + rules + `composes_with`. Adding an action = adding one row; NO code change in either layer (§11.4.28: zero project data).
- **scripts/action_prefix_lib.sh** is the pure, agent-agnostic engine reused by all three consumers (hook, tests, generator) — one tested implementation. Its `apx_expand_prompt` emits a JSON verdict (`expand / noop / escape / ask`) implementing the full grammar: first-non-blank-line anchoring, UPPERCASE-only token, exact " :: " separator, outer-to-inner stacked-prefix recursion, leading-`\` escape, and unknown-token→ASK (§11.4.6 no-guessing / §11.4.66 / §11.4.105 — never invents an expansion, names the closest registered action). The registry path is `$HELIX_ACTION_REGISTRY`-overridable (§11.4.28 decoupling — proven by an alt-registry test).
- **scripts/hooks/action_prefix_expand.sh** (LAYER 2) reads the `UserPromptSubmit` event, runs the library, and — on a registered match — emits `{"hookSpecificOutput": {"hookEventName": "UserPromptSubmit", "additionalContext": "..."} }` (the spec-verified additive contract, confirmed against the official docs this session). No-match / escape → empty stdout (pass-through). Unknown grammar-shaped token → an `additionalContext` clarify note, never an invented expansion. Any internal error FAILS OPEN to pass-through (a prefix system must never reject a user's prompt). It is the §11.4.109 anti-forgetting pattern applied to prompt prefixes — the expansion holds even if model recall lapses.
- **LAYER 1** (`actions/recognition_instruction.md`) is the always-on universal floor: the identical block embedded into `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md` (§11.4.35) so EVERY CLI agent self-applies the prefix even without a hook. It inlines the BACKGROUND expansion so the most-used action works out-of-the-box with no registry-file access. (Embedding into the live carriers is the conductor's §11.4.26 job — left untouched here.)
- **scripts/generate_agent_prefix_commands.sh** turns each registry action into the per-agent slash-command equivalents (`/background {{args}}` for Gemini/Qwen TOML, a Codex prompts/`background.md`) under `actions/generated/` — the LAYER-2 mechanical convenience for agents with generated commands but no pre-submit hook. New action → re-run → its slash command appears everywhere.

- **scripts/install_action_prefix.sh** is the §11.4.75 out-of-the-box install seam: it merges the UserPromptSubmit hook entry into the consuming project's .claude/settings.json (BY REFERENCE — relative constitution/... path when the constitution is a subdir, never copying the hook, per §11.4.80), idempotently (no duplicate on re-run; pre-existing hooks preserved), backs up settings.json first, and runs the generator.
-

3. YAML-reader dependency choice (+ fallback)

- **Primary: python3 + PyYAML** (apx__have_python_yaml gate). Robust, general YAML parsing for validate / list / expansion. Both present on this host (pyyaml-OK).
- **Fallback: an awk/sed reader** (apx__awk_*) used automatically when python3 or PyYAML is absent. It is HONESTLY NARROW — it targets exactly the registry shape this project emits (schema_version:, grammar.prefix_regex:, actions[] with - name: + a expansion: >- folded block scalar or an inline expansion:), documented as such in the source. It is NOT a general YAML parser. Verified byte-identical to the python path for validate / list / expansion / expand / escape / unknown / lowercase, and correct for first-item / last-item / inline / missing actions in a multi-action registry. (One awk-exit-runs-END duplication bug was found and fixed during testing — the parser now emits exactly once via a single END emit point.)

The JSON the library + hook EMIT is produced/escaped by hand (apx__json_escape, apx__emit_json) so no JSON-writer dependency is required; jq is preferred for READING our own emitted JSON with a sed fallback when absent.

4. Parse-clean confirmation + verification summary

- **Parse-clean:** all four scripts pass BOTH sh -n AND bash -n (§11.4.67). (Shebangs honestly declare #!/usr/bin/env bash — the scripts use bash arrays / [[]] / BASH_SOURCE; bash -n is the authoritative gate.)
- **Registry:** parses under PyYAML AND the awk fallback; apx_validate_registry returns 0; schema_version=1, actions=[BACKGROUND].
- **Unit (expander):** U2 exact-match, U3 lowercase-no-match, U4 mid-prose-no-match, U5 wrong-separator-no-match (BACKGROUND::do X, Foo::Bar), U6 escape, U7 stacked-prefix (outer-to-inner), U8 unknown→ASK+closest, U9 empty, U10

leading-blank-lines, U11 expansion-fidelity (byte-equals registry verbatim) — all verified, on BOTH the python and awk paths.

- **Integration (hook):** I1 match→valid additionalContext JSON (correct hookEventName, expansion + residual present), I2 no-match→empty, I3 escape→empty, I4 unknown→clarify-note (no invented expansion, closest=BACKGROUND), I5 no-jq/minimal-PATH parity — all PASS.
 - **Decoupling:** \$HELIX_ACTION_REGISTRY override expands a custom action from an alternate registry (§11.4.28 proof).
 - **Generator:** produces Gemini/Qwen/Codex commands; expansion byte-present in all three (fidelity OK).
 - **Installer:** fresh install, idempotent re-run (1 entry, no dup), preserves a pre-existing PreToolUse guard hook, and emits a RELATIVE by-reference hook path for a real constitution/subdir.
 - **Determinism (§11.4.50):** the hook's output for a fixed prompt is identical across 3 runs (same sha256).
-

5. Demo — apx_expand_prompt

"BACKGROUND :: do X" (input → output)

```
INPUT  : BACKGROUND :: do X
ACTION : BACKGROUND (verdict=expand)
EMITTED: The following prompt that we will provide MUST BE
executed in background
        in parallel with all main work streams using the
subagents-driven
        development approach! All work done MUST PRODUCE rock
solid evidence
        covered with hard physical proof(s) that all done is
working as expected
        and as specified without any false results and without
any bluff!
        <blank line> do X
```

(The EMITTED field is "<expansion>\n\n<residual>"; the residual do X is the actual task the agent then performs.)

6. Honest scope notes (§11.4.6)

- **Could everything be fully implemented?** YES — all 6 requested components were implemented and verified working without any governance-file edit. The LAYER-1 recognition block (which DOES require editing the 4 carriers) was delivered as canonical, embeddable text in actions/recognition_instruction.md rather than applied to the live

CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md — exactly because those are conductor-owned per the task's explicit instruction. The conductor lands them via §11.4.26.

- **Mechanical free-form interception is Claude-Code-only** (the design's honest §11.4.3 boundary): only Claude Code exposes the UserPromptSubmit pre-submit seam. On every other agent the free-form BACKGROUND :: ... form is honoured by LAYER 1 (the agent self-applies via its carrier block), and the generated /background slash command is the LAYER-2 convenience. This split is stated as fact in DESIGN.md §7 and not papered over.
- **The awk fallback is intentionally narrow** to the project's registry shape; python3+PyYAML is the primary, robust path.